Real-Time Intrusion Alert and Active Response System Using Wazuh and Telegram Bot

Group 2: 1.) Muhammad Rafi Fadhil Falah Athallah
2.) Mufid Fata Rifki

Faculty:

Tri Agus Riyadi, S.Kom., M.T.

Class:

4CS1

CEP CCIT FACULTY OF ENGINEERING

UNIVERSITY OF INDONESIA

PROJECT INFORMATION

Project Title: Real-Time Intrusion Alert and Active Response System Using Wazuh and Telegram Bot

Batch Code: 4CS1

Start Date: March 12, 2026

End Date: March 26, 2026

Name of Faculty: Tri Agus Riyadi, S.Kom., M.T.

Names of Network Administrator:

1. Muhammad Rafi Fadhil Falah Athallah
2. Mufid Fata Rifki

UNIVERSITAS INDONESIA
CEP-CCIT
FAKULTAS TEKNIK

CERTIFICATE

This document serves as verification for the report "Real-Time Intrusion Alert and Active Response System Using Wazuh and Telegram Bot" written by Muhammad Rafi Fadhil Falah Athallah and Mufid Fata Rifki. The goal of this project is to complete the CCIT-FTUI course requirement.

ACKNOWLEDGEMENT

As an author and developer, the author would like to acknowledge the finished product of this paper submission "Real-Time Intrusion Alert and Active Response System Using Wazuh and Telegram Bot". This paper explains the author's project on blue teaming about implementing the alert system if there is any attack incoming on the network, the alert then will be sent through push notifications in the Telegram app. Not only that, it also shows the demonstration of various penetration testing tools that can be used to test how the SIEM application called Wazuh will behave and put the choice of the incident process to a security analyst whether to allow the attack to go through or not, in case of false positives alert.

The authors also want to thank the faculty who mentored and helped with this whole project. And even though there are some imperfections, regardless of the imperfection, this imperfection will serve as a greater learning experience for future projects.

Overall, this paper serves the whole concept of this project, by documenting the whole process, concept, process, and the finished product.

Depok, 12 March 2026

Authors

SYSTEM ANALYSIS

Technology, especially computers, plays a big role in human life especially in today's world where the digital world is also connected to our daily drive. However the more technology develops, the more advanced the method to do cyber attack. Therefore, it is why protecting the information for the organizations is the number 1 priority, one of the methods available is by implementing security alert automation program, which means the cybersecurity employee will implement an existing SIEM application called Wazuh and integrate it to messenger application which is Telegram in this case, then the cybersecurity employee have a choice whether to block the incoming attack or not by pressing the button on the message remotely without having to go to the office, offering for more flexibility and faster actions.

SIEM is cybersecurity technology that provides a single, streamlined view of your data insight into security activities, and operational capabilities. In this case, SIEM can send you an insight alert if there are some suspicious activities, like pinging, brute force, SSH access, and many more. There are many SIEM software but in this project, we will use Wazuh for its SIEM software. Wazuh has many SIEM features which the analyst can customize for better security solutions, and one of the most important things for us is to integrate with Telegram bot to automatically send the SIEM alert into Telegram push notifications and send it to analyst contact. We will also conduct a penetration testing to see how the alert will react and given a choice whether to allow or block the source remotely from Telegram, we'll also give many features to have extended information for each of the alerts.

We will provide detailed step by step and techniques for implementing the alert and responding to the alert. We hope from this paper, we can learn more about the anatomy of intrusion conducted by threat actors, this way we can improve our security posture better and prevent any kinds of attack that can be conducted from inside/outside.

PREPARATION

For this project it needs a few preparation to get the attack conducted successfully, we will categorize that into:

Software

1. **Kali Linux**

   This OS will be used to conduct penetration testing.

2. **Ubuntu (LTS)**

   This OS will be used as the acting SIEM software and victim's operating system

3. **VMware**

   This software will be used as the operator between the two operating systems above, in case you have Windows or MacOS installed as the main operating system.

Tools/Platform

1. **nmap**

   Used to enumerate open ports of target IP addresses.

2. **hydra**

   Used to brute force the Victim's machine, in this case, SSH.

3. **Wazuh**

   Platform that acts as a SIEM for intrusion detection.

Target

Note! all of the OS are conducted inside Virtual Machine. In order to make the attack and alert happen, make sure the network adapter is set to host-only network.

1. **Attacker's IP: 192.168.200.128**
2. **Victim's IP: 192.168.200.131**
3. **SIEM's IP: 192.168.200.130**

The Telegram push notification bot requires a python script in order to make it happen, the script will be given throughout the process of this paper.

SETUP

Wazuh Manager Installation

The recommended OS for Wazuh Manager would be Ubuntu, in this case, we'll be using Ubuntu LTS version. Make sure that we have the NAT and Host-Only network activated for each of our virtual machines.

To download Wazuh, run this command on the terminal:

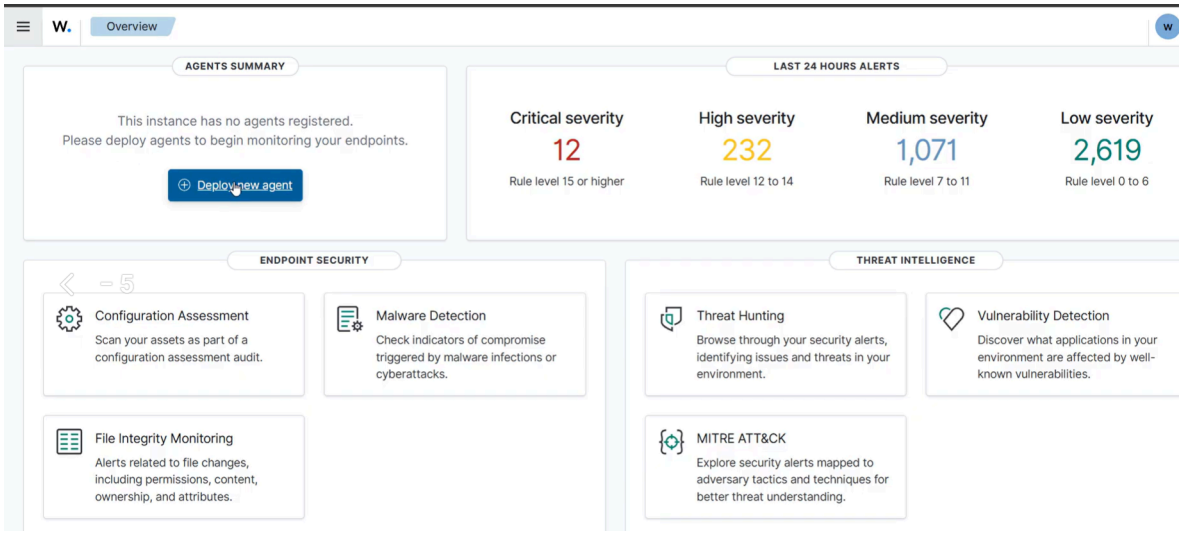
```
$ curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a
```

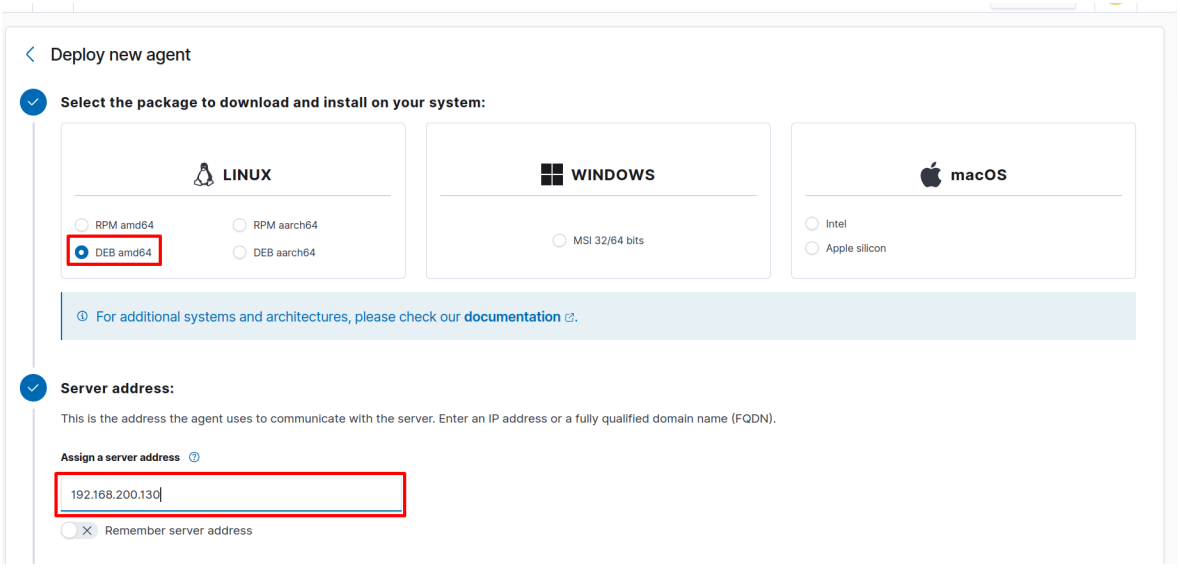
The Wazuh manager will be installed inside, and the system will give you the credentials to access the Wazuh dashboard:

Go to the web browser and enter the link above, in this case our IP is 192.168.200.130, so it will be **https://192.168.200.130:443** go inside and enter the credentials, the dashboard should be opened.

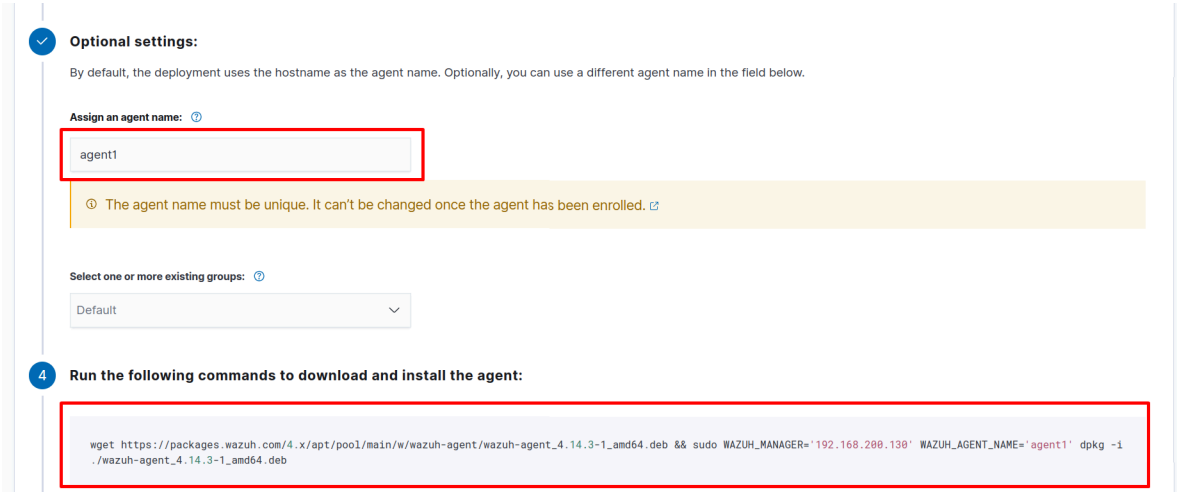
Wazuh Agent Configuration

The meaning of Agent, is a member of the Wazuh that is under its surveillance, meaning that in this scenario, the victim of the attack. Back on the manager dashboard, click on "Deploy New Agent" to add the existing machine.





On the selection we will choose DEB amd64 since we are using the Ubuntu (Debian) VM as the agent. And onto the server address choosing its SIEM IP, we will use 192.168.200.130 as the Server address.



You can type the agent name anything you want but for our case it will be **agent1**, next to the option 4, we will copy the command and enter it on the VM agent. Make sure to run in: Administrator mode (root) & Terminal operating in shell bash.

```
# sudo systemctl daemon-reload
# sudo systemctl enable wazuh-agent
# sudo systemctl start wazuh-agent
```

Next we want to run the above command on the agent virtual machine. The agent should be listed after on the Wazuh dashboard.

SSH Activation

Next is we want to activate or install Secure Shell (SSH) on our Agent virtual machine, this service will be used for our scenario conducting brute force using Hydra and the alert will be listed on the Wazuh and Telegram push notification bot.

Go to the terminal and enter this following command:

```
$ sudo apt install openssh-server
```

After we installed the SSH service, we might want to check its status:

```
$ sudo systemctl status ssh
```

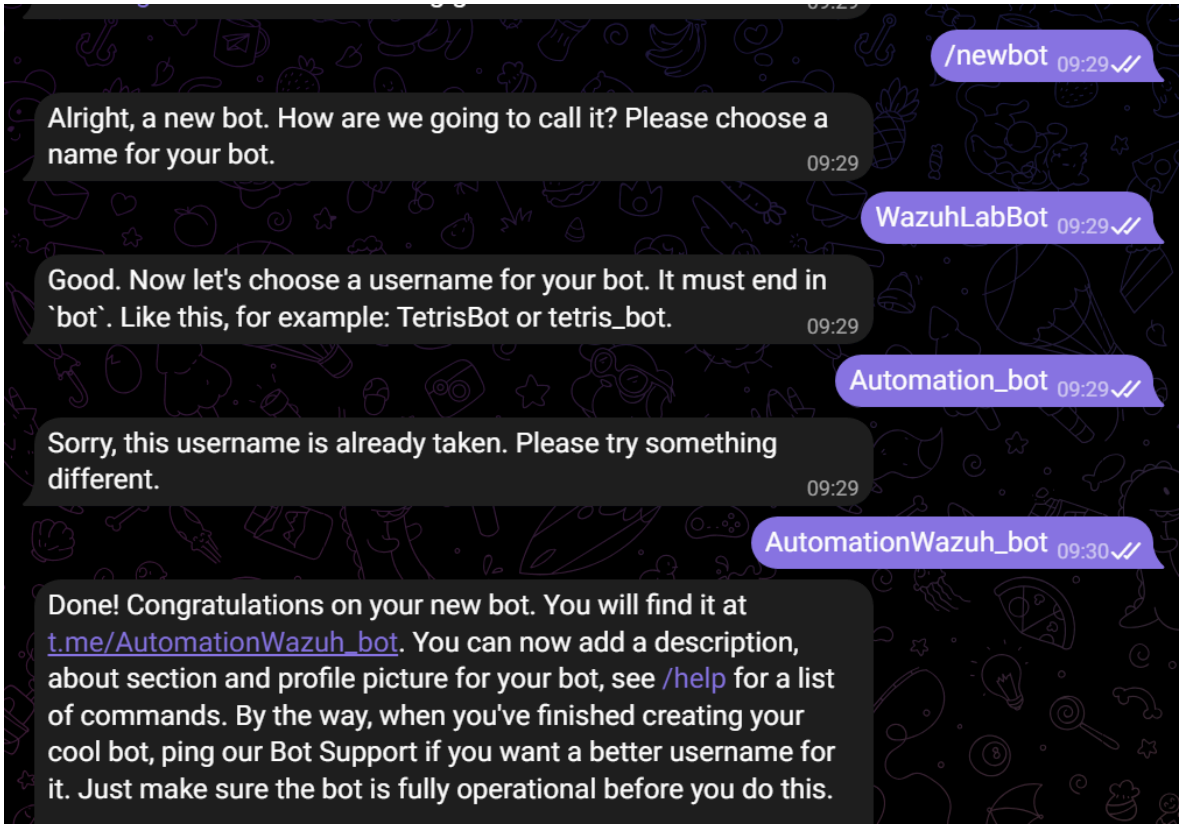
If the output saying its not active, use this command:

```
$ sudo systemctl enable --now ssh
```

The SSH should be active by now, if we want to access the SSH from other users, we can use the following format as the address **`username@IP_ADDRESS.`** The password when we connect to SSH should be the same as the local user password.

CONFIGURATION

After setting up the wazuh, and wazuh agent. The next step is to integrate wazuh with the telegram bot, first thing first is to open telegram and search 'BotFather', and start the conversation.



after sending /start, use the /newbot command to make the bot, type in the name of the bot, and the username. after getting the credential, which is the API key, and then get the chatid from this url `https://api.telegram.org/bot<HTTP-API>/getUpdates`

after getting the credential, make custom-telegram

```
#!/bin/sh

WPYTHON_BIN="framework/python/bin/python3"

SCRIPT_PATH_NAME="$0"

DIR_NAME="$(cd $(dirname ${SCRIPT_PATH_NAME}); pwd -P)"
SCRIPT_NAME="$(basename ${SCRIPT_PATH_NAME})"

case ${DIR_NAME} in
    */active-response/bin | */wodles*)
        if [ -z "${WAZUH_PATH}" ]; then
```

```
            WAZUH_PATH="$(cd ${DIR_NAME}/../..; pwd)"
        fi

        PYTHON_SCRIPT="${DIR_NAME}/${SCRIPT_NAME}.py"
    ;;
    */bin)
        if [ -z "${WAZUH_PATH}" ]; then
            WAZUH_PATH="$(cd ${DIR_NAME}/..; pwd)"
        fi


PYTHON_SCRIPT="${WAZUH_PATH}/framework/scripts/${SCRIPT_NAME}
.py"
    ;;
     */integrations)
        if [ -z "${WAZUH_PATH}" ]; then
            WAZUH_PATH="$(cd ${DIR_NAME}/..; pwd)"
        fi

        PYTHON_SCRIPT="${DIR_NAME}/${SCRIPT_NAME}.py"
    ;;
esac


${WAZUH_PATH}/${WPYTHON_BIN} ${PYTHON_SCRIPT} "$@"
```

This custom-telegram file is the entry point that Wazuh calls directly when an alert is triggered. Wazuh's integration system (wazuh-integratord) can only call executable files directly. When an alert fires, Wazuh looks for the file matching the <name> tag in ossec.conf

custom-telegram.py

```
#!/var/ossec/venv/bin/python

import json

import os

import re

import sys

from datetime import datetime

import requests



bot_token = " " # use your own credentials

chat_id   = " "
```

```
CACHE_DIR = "/tmp/wazuh_alerts"

os.makedirs(CACHE_DIR, exist_ok=True)


excluded_rules: list = []


PRIVATE_RANGES = [

    re.compile(r"^10\."),

    re.compile(r"^172\.(1[6-9]|2[0-9]|3[0-1])\."),

    re.compile(r"^192\.168\."),

    re.compile(r"^127\."),

]




def is_private_ip(ip: str) -> bool:

    return any(p.match(ip) for p in PRIVATE_RANGES)




def get_public_ip() -> str:

        for  url  in  ["https://api.ipify.org",  "https://ifconfig.me/ip",
"https://icanhazip.com"]:

        try:

            resp = requests.get(url, timeout=5)

            if resp.status_code == 200:

                return resp.text.strip()

        except Exception:

            continue

    return "unavailable"




def get_ip_location(ip: str) -> str:
```

```
    if ip == "unavailable" or not ip:
        return "unavailable"
    try:
        resp = requests.get(f"http://ip-api.com/json/{ip}", timeout=5)
        if resp.status_code == 200:
            data = resp.json()
            if data.get("status") == "success":
                city    = data.get("city", "")
                region  = data.get("regionName", "")
                country = data.get("country", "")
                isp     = data.get("isp", "")
                parts   = [p for p in [city, region, country] if p]
                loc     = ", ".join(parts)
                return f"{loc} — {isp}" if isp else loc
    except Exception:
        pass
    return "unavailable"


def escape_markdown_v2(text):
    if not isinstance(text, str):
        text = str(text)
    escape_chars = r"_*[]()~`>#+-=|{}.!"
    return re.sub(r"([%s])" % re.escape(escape_chars), r"\\\1", text)


def build_inline_keyboard(alert_id: str, srcip: str, agent_id: str, rule_id: str, public_ip: str) -> dict:
      lookup_ip = public_ip if (is_private_ip(srcip) and public_ip != "unavailable") else srcip
```

```
    lookup_url = f"https://www.abuseipdb.com/check/{lookup_ip}"


    return {
        "inline_keyboard": [
            [
                {"text": "✅ Acknowledge", "callback_data": f"ack|{rule_id}"[:64]},
                {"text": "📋 Raw JSON", "callback_data": f"json|{alert_id}"[:64]},
                {"text": "📧 Email Report", "callback_data": f"email|{alert_id}"[:64]},
            ],
            [
                {"text": "🔍 Lookup Public IP (AbuseIPDB)", "url": lookup_url},
            ],
        ]
    }



def main():
    if len(sys.argv) < 2:
        print("[ERROR] No alert file path provided.")
        sys.exit(1)

    alert_file = sys.argv[1]
    try:
        with open(alert_file, "r") as f:
            alert = json.load(f)
    except Exception as e:
        print(f"[ERROR] Failed to read or parse alert JSON: {e}")
```

```
        sys.exit(1)


    rule_id = alert.get("rule", {}).get("id", None)

    if rule_id in excluded_rules:

        print(f"[INFO] Skipping excluded rule ID: {rule_id}")

        sys.exit(0)


    data       = alert.get("data", {})

    srcuser    = data.get("srcuser") or data.get("dstuser") or "unknown"

    srcport    = data.get("srcport", "unknown")

    srcip      = data.get("srcip", "unknown")

    agent      = alert.get("agent", {})

    agent_id   = agent.get("id", "000")

    agent_name = agent.get("name", "unknown")

    alert_level   = alert.get("rule", {}).get("level", "unknown")

    rule_id       = alert.get("rule", {}).get("id", "unknown")

        description      =  alert.get("rule",  {}).get("description",  "No
description")

    full_log      = alert.get("full_log", "No full log available")

    timestamp_raw = alert.get("timestamp", "unknown")

        alert_id            =  alert.get("id",  timestamp_raw.replace(":",
"").replace(".", ""))


    try:

        dt = datetime.fromisoformat(timestamp_raw.replace("Z", "+00:00"))

        timestamp = dt.strftime("%Y-%m-%d %H:%M:%S")

    except Exception:

        timestamp = timestamp_raw


    public_ip = get_public_ip() if is_private_ip(srcip) else srcip
```

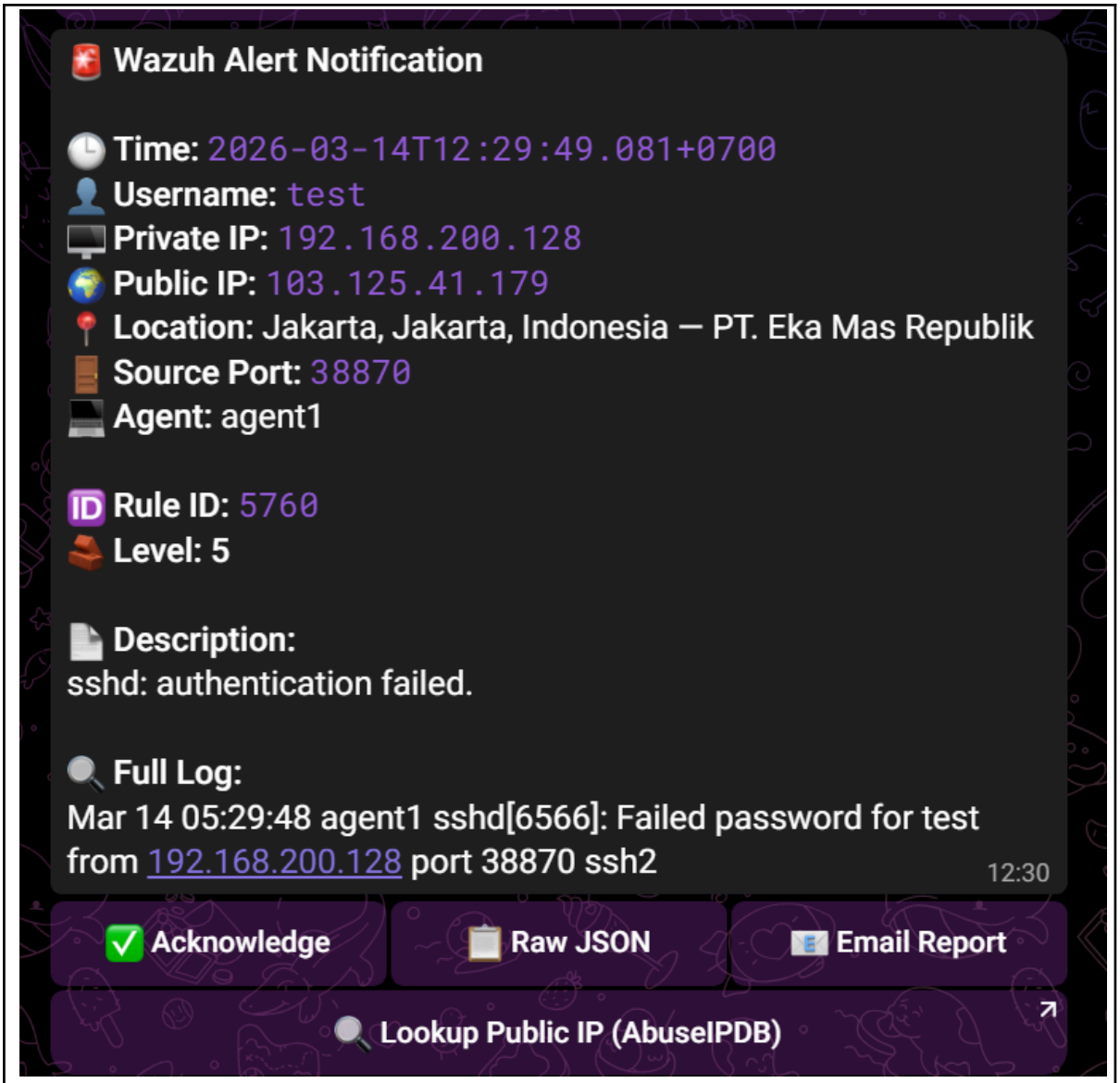
```
    location  = get_ip_location(public_ip)


    alert["_public_ip"] = public_ip

    alert["_location"]  = location


    cache_path = os.path.join(CACHE_DIR, f"{alert_id}.json")

    try:

        with open(cache_path, "w") as f:

            json.dump(alert, f, indent=2)

    except Exception as e:

        print(f"[WARN] Failed to cache alert: {e}")


    # Append to daily summary log

                                                        summary_log            =
f"/tmp/wazuh_alerts/summary_{datetime.now().strftime('%Y-%m-%d')}.jsonl"

    try:

        with open(summary_log, "a") as f:

            f.write(json.dumps({

                "ts": timestamp,

                "rule_id": rule_id,

                "description": description,

                "srcip": srcip,

                "level": alert_level,

                "agent": agent_name,

            }) + "\n")

    except Exception as e:

        print(f"[WARN] Failed to write summary log: {e}")


    message = (

        "*🚨 Wazuh Alert Notification*\n\n"
```

```
        f"🕒 *Time:* `{escape_markdown_v2(timestamp)}`\n"
        f"👤 *Username:* `{escape_markdown_v2(srcuser)}`\n"
        f"🖥️ *Private IP:* `{escape_markdown_v2(srcip)}`\n"
        f"🌍 *Public IP:* `{escape_markdown_v2(public_ip)}`\n"
        f"📍 *Location:* {escape_markdown_v2(location)}\n"
        f"🚪 *Source Port:* `{escape_markdown_v2(srcport)}`\n"
        f"💻 *Agent:* {escape_markdown_v2(agent_name)}\n\n"
        f"🆔 *Rule ID:* `{escape_markdown_v2(rule_id)}`\n"
        f"🧱 *Level:* *{escape_markdown_v2(str(alert_level))}*\n\n"
        f"📄 *Description:*\n{escape_markdown_v2(description)}\n\n"
        f"🔍 *Full Log:*\n{escape_markdown_v2(full_log)}"
    )

    vuln      = alert.get("vulnerability", {})
    cve_id    = vuln.get("cve", "")
    cve_title = vuln.get("title", "")
     cve_url    = f"https://cti.wazuh.com/vulnerabilities/cves/{cve_id}" if
cve_id else ""

    if cve_id:
        message += (
            f"\n\n*🛡️ CVE:* `{escape_markdown_v2(cve_id)}`\n"
            f"*📄 Details:* {escape_markdown_v2(cve_title)}\n"
            f"[🔗 View in CTI]({escape_markdown_v2(cve_url)})"
        )

      keyboard = build_inline_keyboard(alert_id, srcip, agent_id, rule_id,
public_ip)

    url = f"https://api.telegram.org/bot{bot_token}/sendMessage"
```

```
    payload = {

        "chat_id": chat_id,

        "text": message,

        "parse_mode": "MarkdownV2",

        "reply_markup": keyboard,

    }

    response = requests.post(url, json=payload)

    if response.status_code != 200:

              print(f"[ERROR]  Telegram  response:  {response.status_code}  -
{response.text}")




if __name__ == "__main__":

    main()
```

custom-telegram.py is the core integration script that acts as a bridge between Wazuh's alert system and Telegram. It is executed every time Wazuh triggers an alert that meets the configured level threshold.

Wazuh alert fires
↓
custom-telegram (shell wrapper) calls custom-telegram.py
↓
Script reads alert JSON file passed as argument
↓
Enriches alert data (public IP + geolocation)
↓
Caches alert to /tmp/wazuh_alerts/<alert_id>.json
↓
Appends entry to daily summary log
↓
Sends formatted MarkdownV2 message to Telegram via Bot API
with inline keyboard buttons



from the python script, it will make this notification, with all the details

telegram_bot_handler.py

```
#!/usr/bin/env python3

import json

import logging

import os

import re

import smtplib

import subprocess

import time

from collections import Counter
```

```
from datetime import datetime

from email.mime.multipart import MIMEMultipart

from email.mime.text import MIMEText


import requests


# ─── Configuration ──────────────────────────────────────────────────────────

BOT_TOKEN   = "" #use your own credentials

CHAT_ID     = ""


WAZUH_API_URL  = "https://localhost:55000"

WAZUH_USER     = ""

WAZUH_PASSWORD = ""


GMAIL_SENDER    = ""

GMAIL_PASSWORD  = ""

EMAIL_RECIPIENT = ""


CACHE_DIR     = "/tmp/wazuh_alerts"

AGENT_CONTROL = "/var/ossec/bin/agent_control"

DEFAULT_AGENT = "001"

SUMMARY_HOUR  = 20

# ────────────────────────────────────────────────────────────────────────────


TELEGRAM_API = f"https://api.telegram.org/bot{BOT_TOKEN}"

IP_PATTERN   = re.compile(r"^(\d{1,3}\.){3}\d{1,3}$")


logging.basicConfig(

    level=logging.INFO,

    format="%(asctime)s [%(levelname)s] %(message)s",

    datefmt="%Y-%m-%d %H:%M:%S",
```

```
)

log = logging.getLogger(__name__)




# ─── Telegram helpers ──────────────────────────────────────────────────────


def answer_callback(callback_query_id: str, text: str, show_alert: bool = False):

    requests.post(f"{TELEGRAM_API}/answerCallbackQuery", json={

        "callback_query_id": callback_query_id,

        "text": text,

        "show_alert": show_alert,

    })




def edit_reply_markup(chat_id: str, message_id: int, reply_markup: dict | None =
None):

    requests.post(f"{TELEGRAM_API}/editMessageReplyMarkup", json={

        "chat_id": chat_id,

        "message_id": message_id,

        "reply_markup": reply_markup or {},

    })




def send_message(text: str, parse_mode: str = "Markdown"):

    requests.post(f"{TELEGRAM_API}/sendMessage", json={

        "chat_id": CHAT_ID,

        "text": text,

        "parse_mode": parse_mode,

    })
```

```
def send_document(filename: str, content: bytes, caption: str = ""):
    requests.post(f"{TELEGRAM_API}/sendDocument", data={
        "chat_id": CHAT_ID,
        "caption": caption,
    }, files={"document": (filename, content, "application/json")})



# ─── Cache helper ──────────────────────────────────────────────────────────

def load_alert(alert_id: str) -> dict | None:
    path = os.path.join(CACHE_DIR, f"{alert_id}.json")
    try:
        with open(path, "r") as f:
            return json.load(f)
    except Exception as e:
        log.error("Failed to load cached alert %s: %s", alert_id, e)
        return None



# ─── Daily summary ─────────────────────────────────────────────────────────

def load_summary_log(date_str: str) -> list[dict]:
    path = os.path.join(CACHE_DIR, f"summary_{date_str}.jsonl")
    entries = []
    try:
        with open(path, "r") as f:
            for line in f:
                line = line.strip()
                if line:
                    try:
                        entries.append(json.loads(line))
```

```
                    except Exception:
                        pass
    except FileNotFoundError:
        pass
    except Exception as e:
        log.error("Failed to load summary log: %s", e)
    return entries



def build_daily_summary(date_str: str) -> str:
    entries = load_summary_log(date_str)

    if not entries:
        return f"📊 *Daily Summary — {date_str}*\n\nNo alerts recorded today."

    total  = len(entries)
      levels = [int(e.get("level", 0)) for e in entries if str(e.get("level",
"")).isdigit()]
    avg_level = round(sum(levels) / len(levels), 1) if levels else 0
    max_level = max(levels) if levels else 0
    min_level = min(levels) if levels else 0

    ip_counter    = Counter()
    agent_counter = Counter()

    for e in entries:
        srcip = e.get("srcip", "")
        if srcip and srcip != "unknown":
            ip_counter[srcip] += 1
        agent = e.get("agent", "")
        if agent and agent != "unknown":
```

```
            agent_counter[agent] += 1


    top_ips    = ip_counter.most_common(3)

    top_agents = agent_counter.most_common(3)


      top_ip_lines      = "\n".join(f"  • `{ip}` ({count}x)" for ip, count in
top_ips) or "  N/A"

      top_agent_lines = "\n".join(f"  • {agent} ({count}x)" for agent, count in
top_agents) or "  N/A"


    sep = "\u2500" * 28


    return (

        f"📊 *Daily Alert Summary*\n"

        f"📅 Date: `{date_str}`\n"

        f"`{sep}`\n"

        f"🔢 Total Alerts   : `{total}`\n"

        f"📈 Average Level  : `{avg_level}`\n"

        f"🔴 Highest Level  : `{max_level}`\n"

        f"🟢 Lowest Level   : `{min_level}`\n"

        f"`{sep}`\n"

        f"🌐 *Top Attacking IPs:*\n{top_ip_lines}\n"

        f"`{sep}`\n"

        f"💻 *Most Targeted Agents:*\n{top_agent_lines}"

    )



def send_daily_summary(date_str: str = None):

    if not date_str:

        date_str = datetime.now().strftime("%Y-%m-%d")

    summary = build_daily_summary(date_str)

    send_message(summary)
```

```
    log.info("Daily summary sent for %s", date_str)



# ─── Active response ──────────────────────────────────────────────────────────


def run_ar(command: str, srcip: str, agent_id: str) -> bool:

    cmd = [AGENT_CONTROL, "-b", srcip, "-f", f"{command}0", "-u", agent_id]

    try:

        result = subprocess.run(cmd, capture_output=True, text=True, timeout=15)

        output = result.stdout.strip()

          log.info("agent_control [%s %s on %s]: %s", command, srcip, agent_id,
output)

        if "Running active response" in output:

            return True

        log.error("agent_control unexpected output: %s", output)

    except Exception as e:

        log.error("agent_control error: %s", e)

    return False



def run_ar_all_agents(command: str, srcip: str) -> list[str]:

    agents = wazuh_get_agents()

    results = []

    for agent in agents:

        agent_id = agent.get("id", "")

        if not agent_id or agent_id == "000":

            continue

        if run_ar(command, srcip, agent_id):

            results.append(f"{agent.get('name', agent_id)} ({agent_id})")

    return results
```

```
# ─── Wazuh API (agent listing only) ──────────────────────────────────────────


def wazuh_get_token() -> str | None:
    try:
        resp = requests.get(
            f"{WAZUH_API_URL}/security/user/authenticate",
            auth=(WAZUH_USER, WAZUH_PASSWORD),
            verify=False,
            timeout=10,
        )
        if resp.status_code == 200:
            return resp.json().get("data", {}).get("token")
    except Exception as e:
        log.error("Wazuh auth error: %s", e)
    return None



def wazuh_get_agents() -> list[dict]:
    token = wazuh_get_token()
    if not token:
        return []
    try:
        resp = requests.get(
            f"{WAZUH_API_URL}/agents",
            headers={"Authorization": f"Bearer {token}"},
            params={"status": "active", "limit": 100},
            verify=False,
            timeout=10,
        )
        if resp.status_code == 200:
```

```
            return resp.json().get("data", {}).get("affected_items", [])
    except Exception as e:
        log.error("Get agents error: %s", e)
    return []



# ─── Email helper ──────────────────────────────────────────────────────────


def build_html_report(alert: dict) -> str:
    data          = alert.get("data", {})
    rule          = alert.get("rule", {})
    agent         = alert.get("agent", {})
    vuln          = alert.get("vulnerability", {})
    timestamp_raw = alert.get("timestamp", "unknown")
    full_log      = alert.get("full_log", "No full log available")
    public_ip     = alert.get("_public_ip", "N/A")
    location      = alert.get("_location", "N/A")

    try:
        dt = datetime.fromisoformat(timestamp_raw.replace("Z", "+00:00"))
        timestamp = dt.strftime("%Y-%m-%d %H:%M:%S UTC%z")
    except Exception:
        timestamp = timestamp_raw

    cve_id    = vuln.get("cve", "")
    cve_title = vuln.get("title", "")
    cve_block = f"""
        <tr><td><b>CVE ID</b></td><td>{cve_id}</td></tr>
        <tr><td><b>CVE Title</b></td><td>{cve_title}</td></tr>
        <tr><td><b>CVE Link</b></td>
```

```
                                                                                <td><a
href="https://cti.wazuh.com/vulnerabilities/cves/{cve_id}">View                    in
CTI</a></td></tr>

    """ if cve_id else ""



    mitre       = rule.get("mitre", {})

    mitre_block = ""

    if mitre:

        tactics    = ", ".join(mitre.get("tactic", []))

        techniques = ", ".join(mitre.get("technique", []))

        mitre_ids  = ", ".join(mitre.get("id", []))

        mitre_block = f"""

        <tr><td><b>MITRE Tactic</b></td><td>{tactics}</td></tr>

                                <tr><td><b>MITRE    Technique</b></td><td>{techniques}
({mitre_ids})</td></tr>

        """



    return f"""<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8">

<style>

  body {{ font-family: Arial, sans-serif; background: #f4f4f4; padding: 20px; }}

   .container {{ background: #fff; border-radius: 8px; padding: 24px; max-width:
700px; margin: auto; }}

  h2 {{ color: #c0392b; border-bottom: 2px solid #c0392b; padding-bottom: 8px; }}

  h3 {{ color: #2c3e50; margin-top: 20px; }}

  table {{ width: 100%; border-collapse: collapse; margin-top: 8px; }}

  td {{ padding: 8px 12px; border: 1px solid #ddd; vertical-align: top; }}

   td:first-child {{ width: 160px; background: #f9f9f9; font-weight: bold; color:
#555; }}

  .log {{ background: #1e1e1e; color: #d4d4d4; padding: 12px; border-radius: 4px;

                font-family:  monospace;  font-size:  13px;  word-break:  break-all;
white-space: pre-wrap; }}
```

```
    .badge {{ display: inline-block; padding: 3px 10px; border-radius: 12px;
font-size: 13px;

            font-weight: bold; background: #e74c3c; color: white; }}

   .footer {{ margin-top: 24px; font-size: 12px; color: #999; text-align: center;
}}

</style>

</head>

<body>

<div class="container">

  <h2>🚨 Wazuh Security Alert Report</h2>

  <h3>📋 Alert Overview</h3>

  <table>

    <tr><td>Timestamp</td><td>{timestamp}</td></tr>

    <tr><td>Alert ID</td><td>{alert.get("id", "N/A")}</td></tr>

            <tr><td>Level</td><td><span   class="badge">Level   {rule.get("level",
"N/A")}</span></td></tr>

    <tr><td>Rule ID</td><td>{rule.get("id", "N/A")}</td></tr>

    <tr><td>Description</td><td>{rule.get("description", "N/A")}</td></tr>

    {mitre_block}

  </table>

  <h3>💻 Agent Information</h3>

  <table>

    <tr><td>Agent Name</td><td>{agent.get("name", "N/A")}</td></tr>

    <tr><td>Agent ID</td><td>{agent.get("id", "N/A")}</td></tr>

    <tr><td>Agent IP</td><td>{agent.get("ip", "N/A")}</td></tr>

  </table>

  <h3>🌐 Attack Source</h3>

  <table>

    <tr><td>Private IP</td><td>{data.get("srcip", "N/A")}</td></tr>

    <tr><td>Public IP</td><td>{public_ip}</td></tr>

    <tr><td>Location</td><td>{location}</td></tr>

    <tr><td>Source Port</td><td>{data.get("srcport", "N/A")}</td></tr>
```

```
        <tr><td>Username</td><td>{data.get("srcuser")  or  data.get("dstuser")  or
"N/A"}</td></tr>

  </table>

  {f'<h3>🛡️ Vulnerability</h3><table>{cve_block}</table>' if cve_id else ''}

  <h3>🔍 Full Log</h3>

  <div class="log">{full_log}</div>

      <div   class="footer">Generated   by   Wazuh   Telegram   Alert   Bot   —
{datetime.now().strftime("%Y-%m-%d %H:%M:%S")}</div>

</div>

</body>

</html>"""




def send_email_report(alert: dict) -> bool:

    try:

        rule  = alert.get("rule", {})

        data  = alert.get("data", {})

        srcip = data.get("srcip", "unknown")


              subject  =  f"[Wazuh  Alert]  Level  {rule.get('level','?')}  |  Rule
{rule.get('id','?')} | {srcip} — {rule.get('description','')[:60]}"


        msg = MIMEMultipart("alternative")

        msg["Subject"] = subject

        msg["From"]    = GMAIL_SENDER

        msg["To"]      = EMAIL_RECIPIENT

        msg.attach(MIMEText(build_html_report(alert), "html"))


        with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:

            server.login(GMAIL_SENDER, GMAIL_PASSWORD)

            server.sendmail(GMAIL_SENDER, EMAIL_RECIPIENT, msg.as_string())
```

```
        log.info("Email report sent for rule %s", rule.get("id"))
        return True
    except Exception as e:
        log.error("Email send error: %s", e)
        return False



# ─── Command handlers ──────────────────────────────────────────────────────


def handle_command(text: str):
    parts = text.strip().split()
    cmd   = parts[0].lower().split("@")[0]
    args  = parts[1:]

    if cmd == "/help":
        send_message(
            "*🤖 Wazuh Bot Commands*\n\n"
               "`/block <ip> [agent_id]` — Permanently block IP on all agents or
specific agent\n"
            "`/agents` — List all active agents\n"
            "`/summary` — Today's alert summary\n"
            "`/summary <YYYY-MM-DD>` — Summary for specific date\n"
            "`/help` — Show this message"
        )
        return

    if cmd == "/agents":
        agents = wazuh_get_agents()
        if not agents:
            send_message("❌ No active agents found or API unreachable.")
            return
```

```
        lines = ["*💻 Active Agents*\n"]

        for a in agents:

                lines.append(f"`{a.get('id')}` — *{a.get('name')}* ({a.get('ip',
'N/A')})")

        send_message("\n".join(lines))

        return


    if cmd == "/summary":

        date_str = args[0] if args else datetime.now().strftime("%Y-%m-%d")

        send_daily_summary(date_str)

        return


    if cmd == "/block":

        if not args:

               send_message("⚠️ Usage: `/block <ip> [agent_id]`\nExample: `/block
192.168.200.128`")

            return


        srcip    = args[0]

        agent_id = args[1] if len(args) > 1 else None


        if not IP_PATTERN.match(srcip):

            send_message(f"⚠️ Invalid IP address: `{srcip}`")

            return


        if agent_id:

            send_message(f"⏳ Blocking `{srcip}` on agent `{agent_id}`…")

            if run_ar("firewall-block-persistent", srcip, agent_id):

                send_message(f"🔒 *Blocked* `{srcip}` on agent `{agent_id}`")

            else:

                           send_message(f"❌ Failed to block `{srcip}` on agent
`{agent_id}`")
```

```
        else:

            send_message(f"⏳ Blocking `{srcip}` on all agents…")

            results = run_ar_all_agents("firewall-block-persistent", srcip)

            if results:

                send_message(f"🔒 *Blocked* `{srcip}` on:\n" + "\n".join(f"• {a}"
for a in results))

            else:

                send_message(f"❌ Failed to block `{srcip}` on any agent")

        return


        send_message(f"❓ Unknown command: `{cmd}`\nType `/help` for available
commands.")




# ─── Callback handlers ──────────────────────────────────────────────────


def handle_acknowledge(callback_query_id: str, rule_id: str, message_id: int):

    answer_callback(callback_query_id, "✅ Alert acknowledged.")

    edit_reply_markup(CHAT_ID, message_id, {

              "inline_keyboard": [[{"text": "✅ Acknowledged", "callback_data":
"noop"}]]

    })




def handle_raw_json(callback_query_id: str, alert_id: str):

    alert = load_alert(alert_id)

    if not alert:

          answer_callback(callback_query_id, "❌ Alert data not found in cache.",
show_alert=True)

        return


    answer_callback(callback_query_id, "📋 Sending raw JSON…")

    raw = json.dumps(alert, indent=2)
```

```
    if len(raw) <= 4000:
        send_message(f"```json\n{raw}\n```")
    else:
        send_document(
            filename=f"alert_{alert_id}.json",
            content=raw.encode("utf-8"),
                 caption=f"📋 Alert JSON — Rule {alert.get('rule', {}).get('id',
'N/A')}",
        )



def handle_email_report(callback_query_id: str, alert_id: str):
    alert = load_alert(alert_id)
    if not alert:
          answer_callback(callback_query_id, "❌ Alert data not found in cache.",
show_alert=True)
        return

    answer_callback(callback_query_id, "⏳ Sending email report…")
    if send_email_report(alert):
        send_message(f"📧 *Email report sent* to `{EMAIL_RECIPIENT}`")
    else:
          answer_callback(callback_query_id, "❌ Failed to send email. Check bot
logs.", show_alert=True)



def handle_noop(callback_query_id: str):
    answer_callback(callback_query_id, "Already processed.")



# ─── Dispatcher ──────────────────────────────────────────────────────────
```

```
def dispatch(update: dict):

    if "callback_query" in update:

        cq          = update["callback_query"]

        callback_id = cq["id"]

        data        = cq.get("data", "")

        message_id  = cq.get("message", {}).get("message_id")

        parts       = data.split("|")

        action      = parts[0] if parts else ""


        if action == "ack":

                handle_acknowledge(callback_id, parts[1] if len(parts) > 1 else
"unknown", message_id)

        elif action == "json":

            handle_raw_json(callback_id, parts[1] if len(parts) > 1 else "")

        elif action == "email":

            handle_email_report(callback_id, parts[1] if len(parts) > 1 else "")

        elif action == "noop":

            handle_noop(callback_id)

        else:

            answer_callback(callback_id, "Unknown action.")


    elif "message" in update:

        msg     = update["message"]

        text    = msg.get("text", "").strip()

        from_id = str(msg.get("from", {}).get("id", ""))


        if from_id != CHAT_ID:

            log.warning("Ignored message from unauthorized user: %s", from_id)

            return
```

```
        if text.startswith("/"):
            handle_command(text)



# ─── Daily summary scheduler ──────────────────────────────────────────────


def check_daily_summary(last_summary_date: list):
    now   = datetime.now()
    today = now.strftime("%Y-%m-%d")
    if now.hour >= SUMMARY_HOUR and last_summary_date[0] != today:
        last_summary_date[0] = today
        send_daily_summary(today)



# ─── Long-polling loop ────────────────────────────────────────────────────


def poll():
    offset            = None
    last_summary_date = [""]

    log.info("Telegram bot handler started — polling for updates…")

    while True:
        try:
            check_daily_summary(last_summary_date)

                params = {"timeout": 30, "allowed_updates": ["callback_query",
"message"]}
            if offset is not None:
                params["offset"] = offset
```

```
                resp = requests.get(f"{TELEGRAM_API}/getUpdates", params=params,
timeout=35)

            if resp.status_code != 200:

                log.error("getUpdates error: %s", resp.text)

                time.sleep(5)

                continue


            updates = resp.json().get("result", [])

            for update in updates:

                try:

                    dispatch(update)

                except Exception as e:

                    log.error("Dispatch error: %s", e)

                offset = update["update_id"] + 1


        except requests.exceptions.Timeout:

            pass

        except Exception as e:

            log.error("Polling error: %s", e)

            time.sleep(5)




if __name__ == "__main__":

    poll()
```

telegram_bot_handler.py is a long-running daemon that provides the interactive and command-driven side of the Wazuh Telegram bot. While custom-telegram.py only sends one-way notifications, this script listens for user interactions and responds to them in real time.

It runs as a systemd service (wazuh-telegram-bot.service) and stays alive continuously in the background.

systemd keeps daemon alive

↓

Long-polling loop calls Telegram getUpdates every 30s

↓

Two types of incoming updates:

```
├── callback_query (button pressed)
│       ↓
│   dispatch() → specific handler
│
└── message (slash command typed)
        ↓
    handle_command() → specific action
```

Button: ✅ Acknowledge → handle_acknowledge() → edit keyboard to "Acknowledged"

Button: 📋 Raw JSON → handle_raw_json() → send JSON in chat or as file

Button: 📧 Email Report → handle_email_report() → send HTML email via Gmail SMTP

Command: /block <ip> → run_ar() → agent_control firewall-block-persistent

Command: /agents → wazuh_get_agents() → list active agents

Command: /summary → build_daily_summary() → send daily recap message

Command: /help → static help text

ossec.conf

```
<!--

  Wazuh - Manager - Default configuration for ubuntu 24.04

  More info at: https://documentation.wazuh.com

  Mailing list: https://groups.google.com/forum/#!forum/wazuh
```

```
-->


<ossec_config>

  <global>

    <jsonout_output>yes</jsonout_output>

    <alerts_log>yes</alerts_log>

    <logall>no</logall>

    <logall_json>no</logall_json>

    <email_notification>no</email_notification>

    <smtp_server>smtp.example.wazuh.com</smtp_server>

    <email_from>wazuh@example.wazuh.com</email_from>

    <email_to>recipient@example.wazuh.com</email_to>

    <email_maxperhour>12</email_maxperhour>

    <email_log_source>alerts.log</email_log_source>

    <agents_disconnection_time>15m</agents_disconnection_time>

    <agents_disconnection_alert_time>0</agents_disconnection_alert_time>

    <update_check>yes</update_check>

  </global>


  <alerts>

    <log_alert_level>3</log_alert_level>

    <email_alert_level>12</email_alert_level>

  </alerts>


  <!-- Choose between "plain", "json", or "plain,json" for the format of internal
logs -->

  <logging>

    <log_format>plain</log_format>

  </logging>


  <remote>
```

```
  <connection>secure</connection>
  <port>1514</port>
  <protocol>tcp</protocol>
  <queue_size>131072</queue_size>
</remote>

<!-- Policy monitoring -->
<rootcheck>
  <disabled>no</disabled>
  <check_files>yes</check_files>
  <check_trojans>yes</check_trojans>
  <check_dev>yes</check_dev>
  <check_sys>yes</check_sys>
  <check_pids>yes</check_pids>
  <check_ports>yes</check_ports>
  <check_if>yes</check_if>

  <!-- Frequency that rootcheck is executed - every 12 hours -->
  <frequency>43200</frequency>

  <rootkit_files>etc/rootcheck/rootkit_files.txt</rootkit_files>
  <rootkit_trojans>etc/rootcheck/rootkit_trojans.txt</rootkit_trojans>

  <skip_nfs>yes</skip_nfs>

  <ignore>/var/lib/containerd</ignore>
  <ignore>/var/lib/docker/overlay2</ignore>
</rootcheck>

<wodle name="cis-cat">
  <disabled>yes</disabled>
```

```
  <timeout>1800</timeout>
  <interval>1d</interval>
  <scan-on-start>yes</scan-on-start>

  <java_path>wodles/java</java_path>
  <ciscat_path>wodles/ciscat</ciscat_path>
</wodle>

<!-- Osquery integration -->
<wodle name="osquery">
  <disabled>yes</disabled>
  <run_daemon>yes</run_daemon>
  <log_path>/var/log/osquery/osqueryd.results.log</log_path>
  <config_path>/etc/osquery/osquery.conf</config_path>
  <add_labels>yes</add_labels>
</wodle>

<!-- System inventory -->
<wodle name="syscollector">
  <disabled>no</disabled>
  <interval>1h</interval>
  <scan_on_start>yes</scan_on_start>
  <hardware>yes</hardware>
  <os>yes</os>
  <network>yes</network>
  <packages>yes</packages>
  <ports all="yes">yes</ports>
  <processes>yes</processes>
  <users>yes</users>
  <groups>yes</groups>
  <services>yes</services>
```

```
  <browser_extensions>yes</browser_extensions>


  <!-- Database synchronization settings -->
  <synchronization>
    <max_eps>10</max_eps>
  </synchronization>
</wodle>


<sca>
  <enabled>yes</enabled>
  <scan_on_start>yes</scan_on_start>
  <interval>12h</interval>
  <skip_nfs>yes</skip_nfs>
</sca>


<vulnerability-detection>
  <enabled>yes</enabled>
  <index-status>yes</index-status>
  <feed-update-interval>60m</feed-update-interval>
</vulnerability-detection>


<indexer>
  <enabled>yes</enabled>
  <hosts>
    <host>https://127.0.0.1:9200</host>
  </hosts>
  <ssl>
    <certificate_authorities>
      <ca>/etc/filebeat/certs/root-ca.pem</ca>
    </certificate_authorities>
    <certificate>/etc/filebeat/certs/wazuh-server.pem</certificate>
```

```
    <key>/etc/filebeat/certs/wazuh-server-key.pem</key>
  </ssl>
</indexer>

<!-- File integrity monitoring -->
<syscheck>
  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <scan_on_start>yes</scan_on_start>

  <!-- Generate alert when new file detected -->
  <alert_new_files>yes</alert_new_files>

  <!-- Don't ignore files that change more than 'frequency' times -->
  <auto_ignore frequency="10" timeframe="3600">no</auto_ignore>

  <!-- Directories to check  (perform all possible verifications) -->
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories>/bin,/sbin,/boot</directories>

  <!-- Files/directories to ignore -->
  <ignore>/etc/mtab</ignore>
  <ignore>/etc/hosts.deny</ignore>
  <ignore>/etc/mail/statistics</ignore>
  <ignore>/etc/random-seed</ignore>
  <ignore>/etc/random.seed</ignore>
  <ignore>/etc/adjtime</ignore>
  <ignore>/etc/httpd/logs</ignore>
```

```
  <ignore>/etc/utmpx</ignore>

  <ignore>/etc/wtmpx</ignore>

  <ignore>/etc/cups/certs</ignore>

  <ignore>/etc/dumpdates</ignore>

  <ignore>/etc/svc/volatile</ignore>


  <!-- File types to ignore -->

  <ignore type="sregex">.log$|.swp$</ignore>


  <!-- Check the file, but never compute the diff -->

  <nodiff>/etc/ssl/private.key</nodiff>


  <skip_nfs>yes</skip_nfs>

  <skip_dev>yes</skip_dev>

  <skip_proc>yes</skip_proc>

  <skip_sys>yes</skip_sys>


  <!-- Nice value for Syscheck process -->

  <process_priority>10</process_priority>


  <!-- Maximum output throughput -->

  <max_eps>50</max_eps>


  <!-- Database synchronization settings -->

  <synchronization>

    <enabled>yes</enabled>

    <interval>5m</interval>

    <max_eps>10</max_eps>

  </synchronization>

</syscheck>
```

```
<!-- Active response -->
<global>
  <white_list>127.0.0.1</white_list>
  <white_list>^localhost.localdomain$</white_list>
  <white_list>127.0.0.53</white_list>
</global>

<command>
  <name>disable-account</name>
  <executable>disable-account</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>restart-wazuh</name>
  <executable>restart-wazuh</executable>
</command>

<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>firewall-block-persistent</name>
  <executable>firewall-block-persistent</executable>
  <timeout_allowed>no</timeout_allowed>
</command>

<command>
```

```
  <name>firewall-unblock</name>

  <executable>firewall-unblock</executable>

  <timeout_allowed>no</timeout_allowed>

</command>


<command>

  <name>host-deny</name>

  <executable>host-deny</executable>

  <timeout_allowed>yes</timeout_allowed>

</command>


<command>

  <name>route-null</name>

  <executable>route-null</executable>

  <timeout_allowed>yes</timeout_allowed>

</command>


<command>

  <name>win_route-null</name>

  <executable>route-null.exe</executable>

  <timeout_allowed>yes</timeout_allowed>

</command>


<command>

  <name>netsh</name>

  <executable>netsh.exe</executable>

  <timeout_allowed>yes</timeout_allowed>

</command>



<active-response>
```

```
    <command>firewall-drop</command>

    <location>all</location>

    <rules_id>5760</rules_id>

    <timeout>60</timeout>

  </active-response>


  <active-response>

    <command>firewall-block-persistent</command>

    <location>all</location>

  </active-response>


  <active-response>

    <command>firewall-unblock</command>

    <location>all</location>

  </active-response>


  <!-- Log analysis -->

  <localfile>

    <log_format>command</log_format>

    <command>df -P</command>

    <frequency>360</frequency>

  </localfile>


  <localfile>

    <log_format>full_command</log_format>

        <command>netstat  -tulpn  |  sed  's/\([[:alnum:]]\+\)\  \+[[:digit:]]\+\
\+[[:digit:]]\+\        \+\(.*\):\([[:digit:]]*\)\        \+\([0-9\.\:\*]\+\).\+\
\([[:digit:]]*\/[[:alnum:]\-]*\).*/\1 \2 == \3 == \4 \5/' | sort -k 4 -g | sed
's/ == \(.*\) ==/:\1/' | sed 1,2d</command>

    <alias>netstat listening ports</alias>

    <frequency>360</frequency>

  </localfile>
```

```
<localfile>
  <log_format>full_command</log_format>
  <command>last -n 20</command>
  <frequency>360</frequency>
</localfile>

<ruleset>
  <!-- Default ruleset -->
  <decoder_dir>ruleset/decoders</decoder_dir>
  <rule_dir>ruleset/rules</rule_dir>
  <rule_exclude>0215-policy_rules.xml</rule_exclude>
  <list>etc/lists/audit-keys</list>
  <list>etc/lists/amazon/aws-eventnames</list>
  <list>etc/lists/security-eventchannel</list>
  <list>etc/lists/malicious-ioc/malware-hashes</list>
  <list>etc/lists/malicious-ioc/malicious-ip</list>
  <list>etc/lists/malicious-ioc/malicious-domains</list>

  <!-- User-defined ruleset -->
  <decoder_dir>etc/decoders</decoder_dir>
  <rule_dir>etc/rules</rule_dir>
</ruleset>

<rule_test>
  <enabled>yes</enabled>
  <threads>1</threads>
  <max_sessions>64</max_sessions>
  <session_timeout>15m</session_timeout>
</rule_test>
```

```
  <integration>
      <name>custom-telegram</name>
      <level>4</level>
      <hook_url>https://api.telegram.org/<API_KEY>/sendMessage</hook_url>
      <alert_format>json</alert_format>
  </integration>

<!-- Configuration for wazuh-authd -->
<auth>
  <disabled>no</disabled>
  <port>1515</port>
  <use_source_ip>no</use_source_ip>
  <purge>yes</purge>
  <use_password>no</use_password>
  <ciphers>HIGH:!ADH:!EXP:!MD5:!RC4:!3DES:!CAMELLIA:@STRENGTH</ciphers>
  <!-- <ssl_agent_ca></ssl_agent_ca> -->
  <ssl_verify_host>no</ssl_verify_host>
  <ssl_manager_cert>etc/sslmanager.cert</ssl_manager_cert>
  <ssl_manager_key>etc/sslmanager.key</ssl_manager_key>
  <ssl_auto_negotiate>no</ssl_auto_negotiate>
</auth>

<cluster>
  <name>wazuh</name>
  <node_name>node01</node_name>
  <node_type>master</node_type>
  <key></key>
  <port>1516</port>
  <bind_addr>0.0.0.0</bind_addr>
  <nodes>
      <node>NODE_IP</node>
```

```
    </nodes>
    <hidden>no</hidden>
    <disabled>yes</disabled>
  </cluster>

</ossec_config>

<ossec_config>
  <localfile>
    <log_format>journald</log_format>
    <location>journald</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/ossec/logs/active-responses.log</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/dpkg.log</location>
  </localfile>
</ossec_config>

<ossec_config>
  <global>
    <jsonout_output>yes</jsonout_output>
    <alerts_log>yes</alerts_log>
    <logall>yes</logall>
    <logall_json>yes</logall_json>
  </global>
```

```
</ossec_config>
```

ossec.conf is the main configuration file for the Wazuh Manager, located at /var/ossec/etc/ossec.conf. It is an XML-based file that controls every aspect of Wazuh's behavior — from log collection and file integrity monitoring to active response and third-party integrations. In this project, it was modified from the default Ubuntu 24.04 template to support the Telegram integration and automated IP blocking.

local_rules.xml

```
<group name="local,syslog,sshd,">
  <rule id="100011" level="7" overwrite="yes">
    <if_sid>5715</if_sid>
    <description>sshd: Successful SSH login with elevated level.</description>
    <group>authentication_success,pci_dss_10.2.5,</group>
  </rule>
</group>

<group name="local,audit,suspicious,">
  <rule id="100040" level="8">
    <if_sid>80792</if_sid>
    <field name="audit.key">suspicious_cmd</field>
              <description>Suspicious   command   executed:   $(audit.exe)   by
$(audit.auid)</description>
    <group>audit,suspicious_command</group>
    <mitre>
      <id>T1059</id>
    </mitre>
  </rule>

  <rule id="100041" level="10">
    <if_sid>80792</if_sid>
    <field name="audit.key">sensitive_file</field>
             <description>Sensitive   file   accessed:   $(audit.file.name)   by
$(audit.auid)</description>
    <group>audit,sensitive_file_access</group>
    <mitre>
      <id>T1003</id>
    </mitre>
  </rule>

  <rule id="100042" level="7">
    <if_sid>80792</if_sid>
    <field name="audit.key">sudo_commands</field>
                <description>Command    executed    with    root    privileges:
$(audit.exe)</description>
    <group>audit,sudo_usage</group>
    <mitre>
      <id>T1548.003</id>
    </mitre>
  </rule>
</group>
```

this file is for additional rules for the telegram notification

wazuh.rules

```
# Monitor sudo usage
-a always,exit -F arch=b64 -F euid=0 -S execve -k sudo_commands

# Monitor suspicious commands
-a always,exit -F arch=b64 -F exe=/usr/bin/whoami -k suspicious_cmd
-a always,exit -F arch=b64 -F exe=/usr/bin/id -k suspicious_cmd
-a always,exit -F arch=b64 -F exe=/bin/cat -k suspicious_cmd
-a always,exit -F arch=b64 -F exe=/usr/bin/passwd -k suspicious_cmd
```

```
# Monitor sensitive file access
-w /etc/passwd -p r -k sensitive_file
-w /etc/shadow -p r -k sensitive_file
-w /etc/sudoers -p r -k sensitive_file
-w /root -p r -k sensitive_file
```

wazuh.rules is located at /etc/audit/rules.d/wazuh.rules on the agent (agent1). It defines rules for the Linux Audit Daemon (auditd), which operates at the kernel level and intercepts system calls before they complete. This provides a layer of visibility that goes far deeper than log file monitoring.

wazuh-telegram-bot.service

```
[Unit]
Description=Wazuh Telegram Bot Handler (Inline Button Callbacks)
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=root
ExecStart=/var/ossec/framework/python/bin/python3
/var/ossec/integrations/telegram_bot_handler.py
Restart=on-failure
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

wazuh-telegram-bot.service is a systemd unit file located at /etc/systemd/system/wazuh-telegram-bot.service. It manages telegram_bot_handler.py as a persistent background service.

firewall-block-persistent

```
#!/usr/bin/env python3
import json
import subprocess
import sys


def run(cmd):
    subprocess.run(cmd, shell=True, check=False)


def main():
    try:
        first_line = sys.stdin.readline()
        data = json.loads(first_line)
    except Exception:
        sys.exit(1)

    command = data.get("command", "")
    alert   = data.get("parameters", {}).get("alert", {})
    srcip   = alert.get("data", {}).get("srcip", "")

    if not srcip:
```

```
        keys  = data.get("parameters", {}).get("keys", [])
        srcip = keys[0] if keys else ""

    if not srcip:
        sys.exit(1)

    if command == "add":
        check_keys = json.dumps({
            "version": 1,
                        "origin":  {"name":  "firewall-block-persistent",  "module":
"active-response"},
            "command": "check_keys",
            "parameters": {"keys": [srcip]}
        })
        print(check_keys, flush=True)

        try:
            cont_line = sys.stdin.readline()
            cont_data = json.loads(cont_line)
            if cont_data.get("command") != "continue":
                sys.exit(0)
        except Exception:
            sys.exit(0)

          run(f"iptables -C INPUT -s {srcip} -j DROP 2>/dev/null || iptables -I
INPUT -s {srcip} -j DROP")
        run("iptables-save > /etc/iptables/rules.v4")

    elif command == "delete":
        run(f"iptables -D INPUT -s {srcip} -j DROP 2>/dev/null")
        run("iptables-save > /etc/iptables/rules.v4")


if __name__ == "__main__":
    main()
```

firewall-block-persistent is a custom active response script located at /var/ossec/active-response/bin/ on both the manager and agent. Unlike the built-in firewall-drop which creates temporary iptables rules that are cleared on timeout or reboot, this script creates permanent rules that survive system restarts.

script and services directory path
/etc/systemd/system/wazuh-telegram-bot.service
/var/ossec/etc/ossec.conf
/var/ossec/etc/rules/local_rules.xml
/var/ossec/etc/decoders/local_decoder.xml
/var/ossec/integrations/custom-telegram
/var/ossec/integrations/custom-telegram.py
/var/ossec/integrations/telegram_bot_handler.py
/var/ossec/active-response/bin/firewall-block-persistent

ATTACK SIMULATION

for the simulation, it's a SSH brute force detection and prevention through telegram interactive features. In this case the attacker 192.168.200.128 (kali linux) will attack 192.168.200.131 (agent1 / ubuntu). in this case the credential for the agent1 lab is **test:wrongpassword123**

```
pxilg@kali  ~/Documents/iseng/lab
└─$ ping 192.168.200.131
PING 192.168.200.131 (192.168.200.131) 56(84) bytes of data.
64 bytes from 192.168.200.131: icmp_seq=1 ttl=64 time=0.522 ms
64 bytes from 192.168.200.131: icmp_seq=2 ttl=64 time=0.471 ms
64 bytes from 192.168.200.131: icmp_seq=3 ttl=64 time=0.312 ms
^C
--- 192.168.200.131 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2030ms
rtt min/avg/max/mdev = 0.312/0.435/0.522/0.089 ms
```

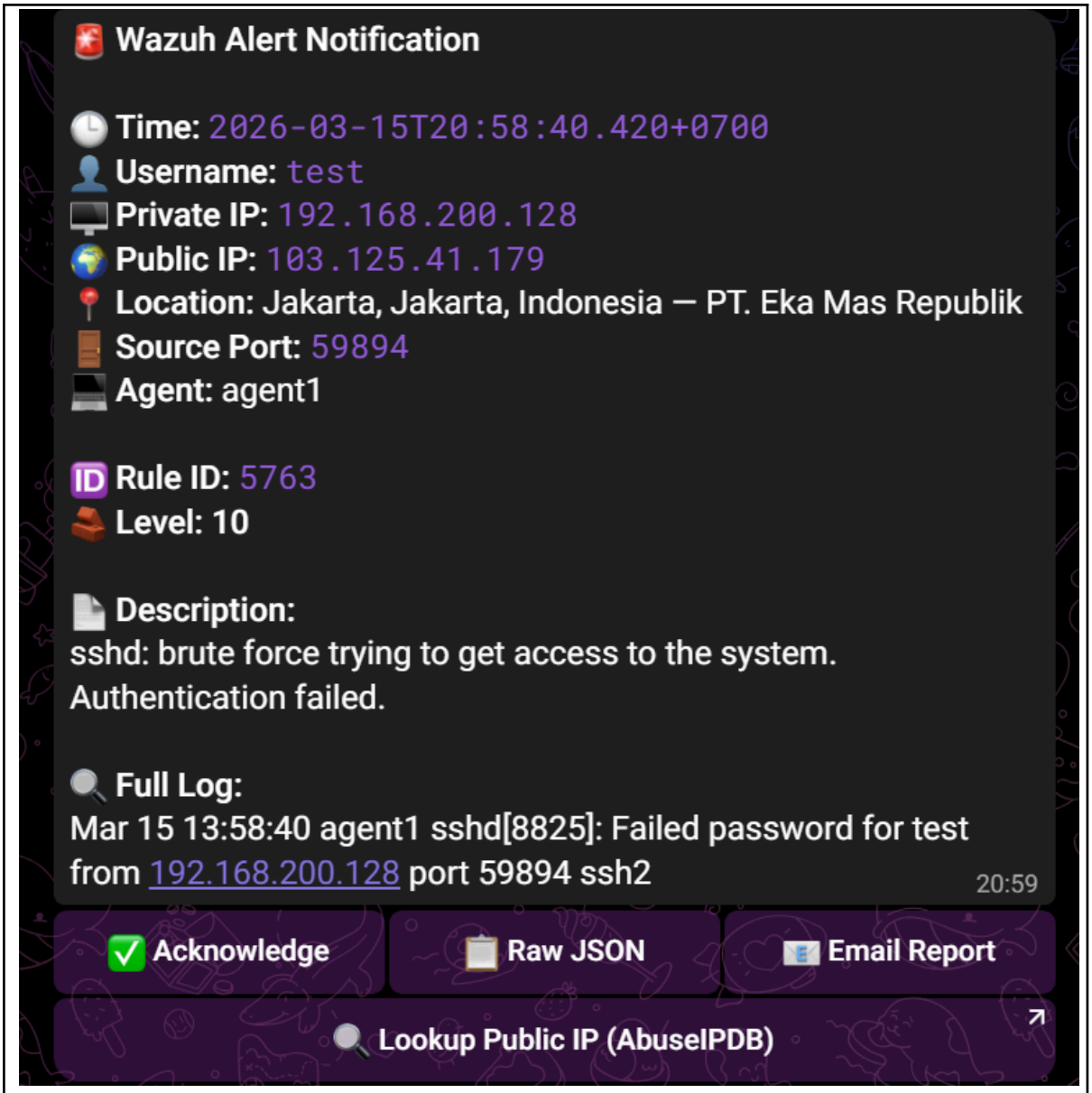
first try to check the connection from attacker to target, and then try to nmap for opening

```
└─$ nmap 192.168.200.131
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-15 20:56 +0700
Nmap scan report for 192.168.200.131
Host is up (0.00014s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
MAC Address: 00:0C:29:A7:DF:D5 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.72 seconds
```

in this scenario it will detect a brute force with hydra tool.

```
└─$ hydra -l test -P /usr/share/wordlists/rockyou.txt ssh://192.168.200.131 -t 8 -I -VV
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-
binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-15 20:58:36
[WARNING] Restorefile (ignored ...) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 8 tasks per 1 server, overall 8 tasks, 14344399 login tries (l:1/p:14344399), ~1793050 tries per task
[DATA] attacking ssh://192.168.200.131:22/
[ATTEMPT] target 192.168.200.131 - login "test" - pass "123456" - 1 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "12345" - 2 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "123456789" - 3 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "password" - 4 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "iloveyou" - 5 of 14344399 [child 4] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "princess" - 6 of 14344399 [child 5] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "1234567" - 7 of 14344399 [child 6] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "rockyou" - 8 of 14344399 [child 7] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "12345678" - 9 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "abc123" - 10 of 14344399 [child 6] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "nicole" - 11 of 14344399 [child 4] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "daniel" - 12 of 14344399 [child 5] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "babygirl" - 13 of 14344399 [child 7] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "monkey" - 14 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "lovely" - 15 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.200.131 - login "test" - pass "jessica" - 16 of 14344399 [child 3] (0/0)
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume session.

pxilg@kali  ~/Documents/iseng/lab                                  ✓ 11s  0.07  20:58:47
└─$ ping 192.168.200.131
PING 192.168.200.131 (192.168.200.131) 56(84) bytes of data.
^C
--- 192.168.200.131 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3068ms
```

The wazuh detected the brute force attempt on the agent, as it shows on the ping after the hydra execution, the attacker no longer can connect to target ip, because of the active response feature that configured in ossec.conf, with a timeout of 60 seconds.



the wazuh detects it and forward it into the telegram chat bot, the chat provided a detail information from time, username, private ip, public ip, location, source port, agent, and the description of the attack, and it also has a features, it can gives raw json detail, email report, and a lookup public IP.

JSON Today


```
{
  "timestamp": "2026-03-15T20:58:40.420+0700",
  "rule": {
    "level": 10,
    "description": "sshd: brute force trying to get
access to the system. Authentication failed.",
    "id": "5763",
    "mitre": {
      "id": [
        "T1110"
      ],
      "tactic": [
        "Credential Access"
      ],
      "technique": [
        "Brute Force"
      ]
    },
    "frequency": 8,
    "firedtimes": 1,
    "mail": false,
    "groups": [
      "syslog",
      "sshd",
      "authentication_failures"
    ],
    "gdpr": [
      "IV_35.7.d",
      "IV_32.2"
    ],
    "hipaa": [
      "164.312.b"
    ],
    "nist_800_53": [
      "SI.4",
      "AU.14",
      "AC.7"
    ],
    "pci_dss": [
      "11.4",
      "10.2.4",
      "10.2.5"
    ]
```

🚨 Wazuh Security Alert Report

📋 Alert Overview

Timestamp	2026-03-15T20:58:40.420+0700
Alert ID	1773583120.490424
Level	Level 10
Rule ID	5763
Description	sshd: brute force trying to get access to the system. Authentication failed.
MITRE Tactic	Credential Access
MITRE Technique	Brute Force (T1110)

💻 Agent Information

Agent Name	agent1
Agent ID	001
Agent IP	192.168.200.131

🌐 Attack Source

Private IP	192.168.200.128
Public IP	103.125.41.179
Location	Jakarta, Jakarta, Indonesia — PT. Eka Mas Republik
Source Port	59894
Username	test

🔍 Full Log

```
Mar 15 13:58:40 agent1 sshd[8825]: Failed password for test from 192.168.200.128 port 59894 ss
h2
```

Generated by Wazuh Telegram Alert Bot — 2026-03-15 21:03:37

AbuseIPDB Report IP Bulk Checker Bulk Reporter Pricing Docs IP Utilities Contact More Login Sign Up

AbuseIPDB » *103.125.41.179*

Check an IP Address, Domain Name, Subnet, or ASN
e.g. **103.125.41.179**, **microsoft.com**, **5.188.10.0/24**, or **AS15169**

103.125.41.179 CHECK

feedback

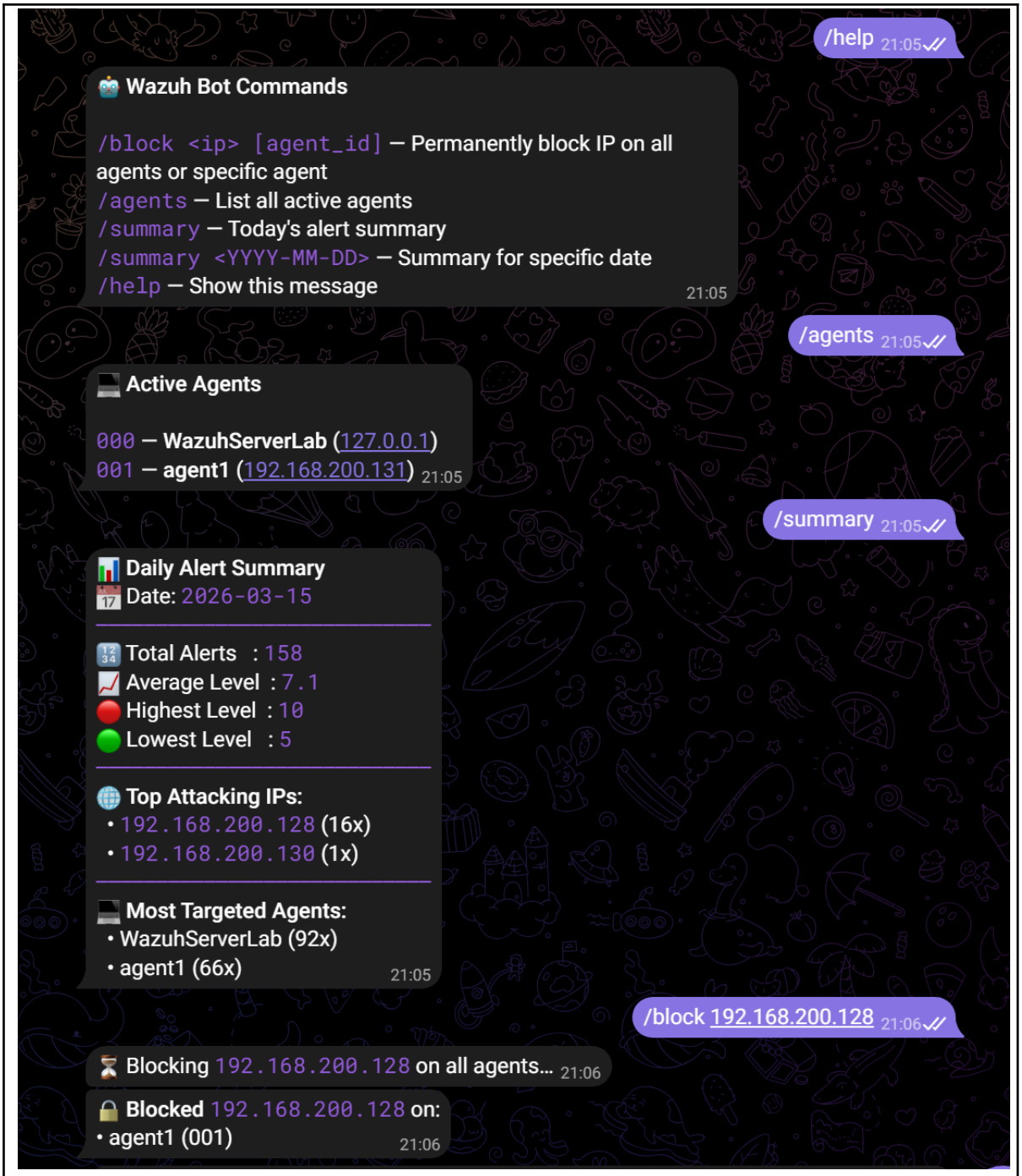
103.125.41.179 was not found in our database

ISP	PT. Eka Mas Republik
Usage Type	Fixed Line ISP
ASN	Unknown
Hostname(s)	host-103-125-41-179.myrepublic.co.id
Domain Name	myrepublic.co.id
Country	Indonesia
City	South Tangerang, Banten

IP info including ISP, Usage Type, and Location provided by IPInfo. Updated biweekly.

REPORT IP WHOIS SEARCH

IP Abuse Reports for **103.125.41.179**:



and with /help command, this bot can do additional features, and also permanent block

```
root@agent1:/home/wazuhserver# iptables -L INPUT
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       all  --  192.168.200.128      anywhere
```

blocked using iptables. and to unblock it should be manually from the agent1 using iptables command.

REQUIREMENTS

Hardware:
ASUS Laptop
Operating System:

1. Windows 11 Pro 64 bit
2. Kali Linux
3. Ubuntu LTS

Software:

1. VMware Workstation
2. Ms. Word
3. Google Chrome

PROJECT FILE DETAILS

No	Filename	Remarks
1.	project_kl2_paper_.pdf	Paper file
2.	project_kl2_presentation.pptx	Presentation File